Security & Governance in Architecture

Mastering Software Architecture Patterns and System Design

Rahul Rajat Singh

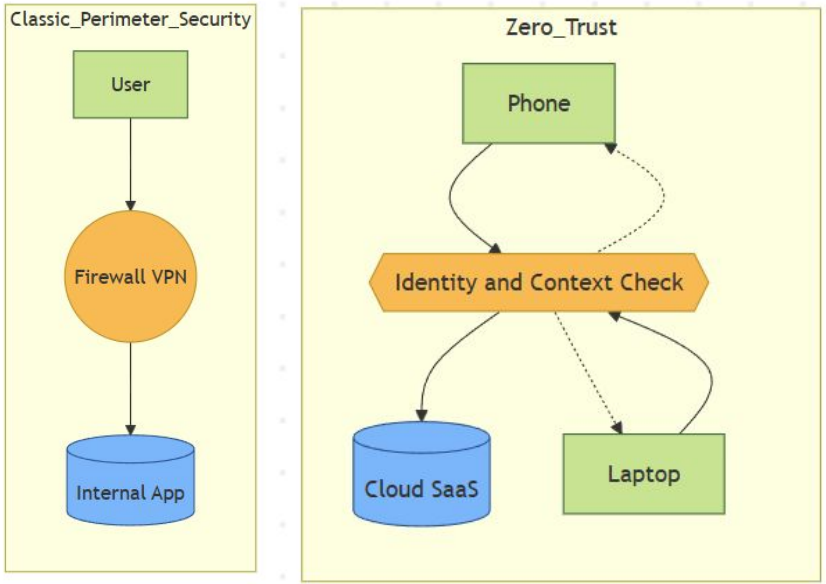
Security Principles & Zero Trust

Security & Governance in Architecture

Mastering Software Architecture Patterns and System Design

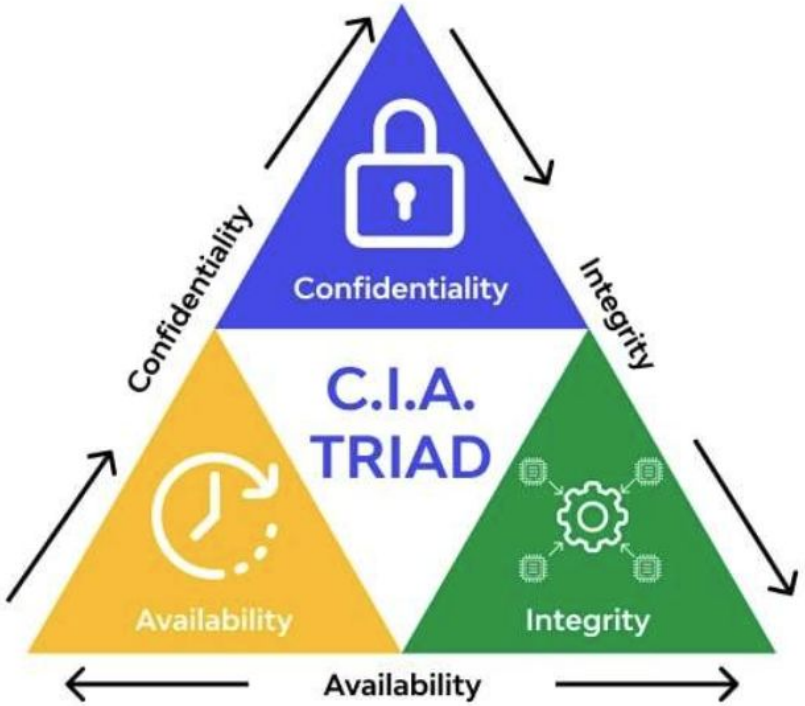
Introduction: Evolving Security Paradigms

- Traditional perimeter-based security is no longer enough
- Rise of cloud, remote work, and BYOD changed threat landscape
- Zero Trust: “Never trust, always verify”



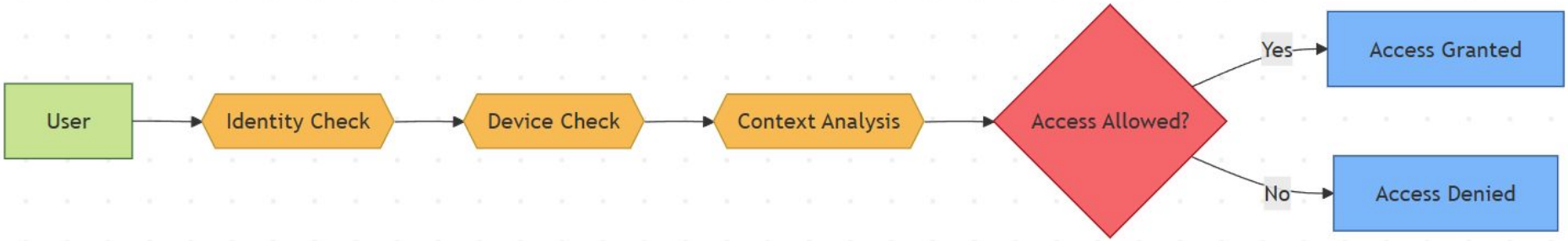
Core Security Principles

- Confidentiality, Integrity, Availability (CIA triad)
- Principle of Least Privilege
- Defense in Depth



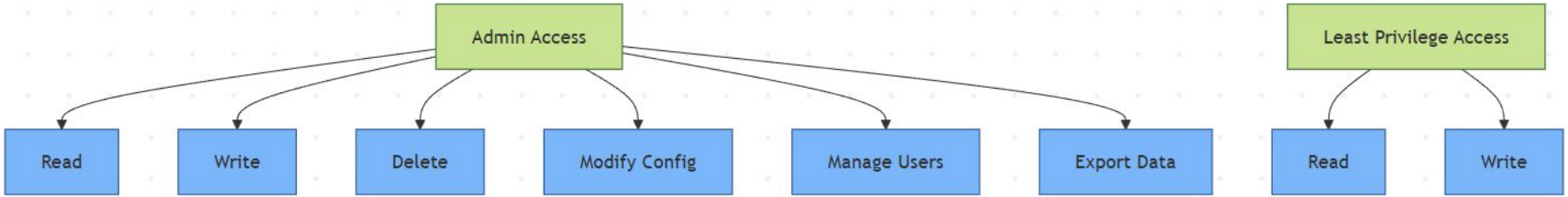
What is Zero Trust?

- No implicit trust, even inside the network
- Continuous authentication and authorization
- Based on identity, context, and device health



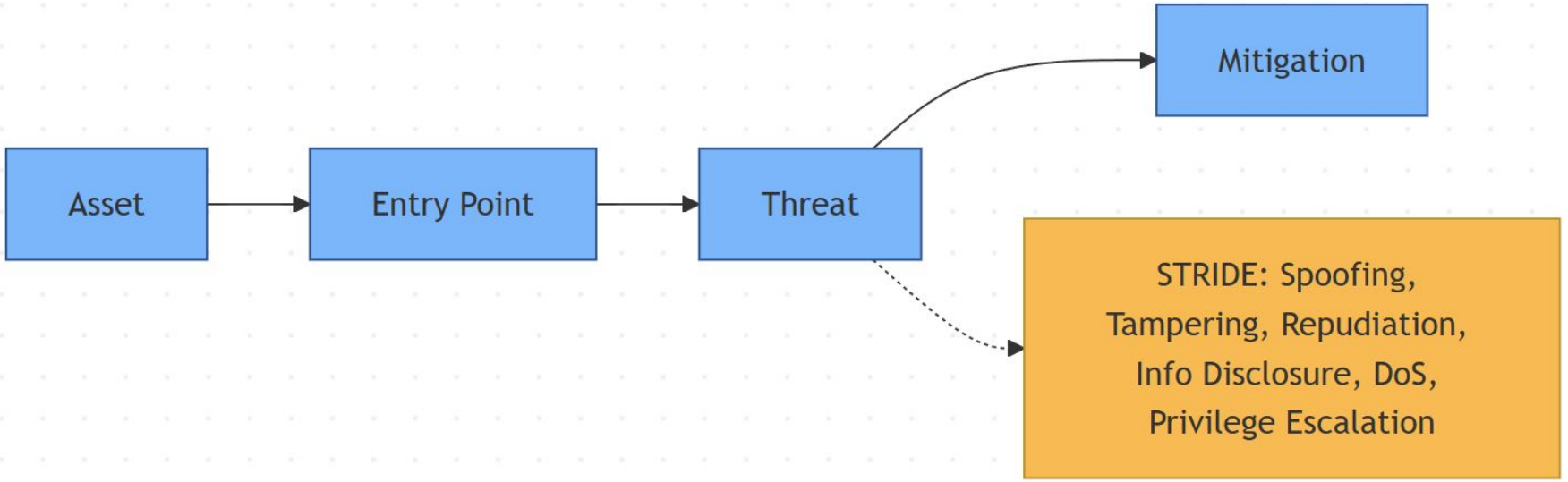
Principle of Least Privilege

- Users/apps get only the access needed
- Reduces attack surface
- Requires fine-grained access controls



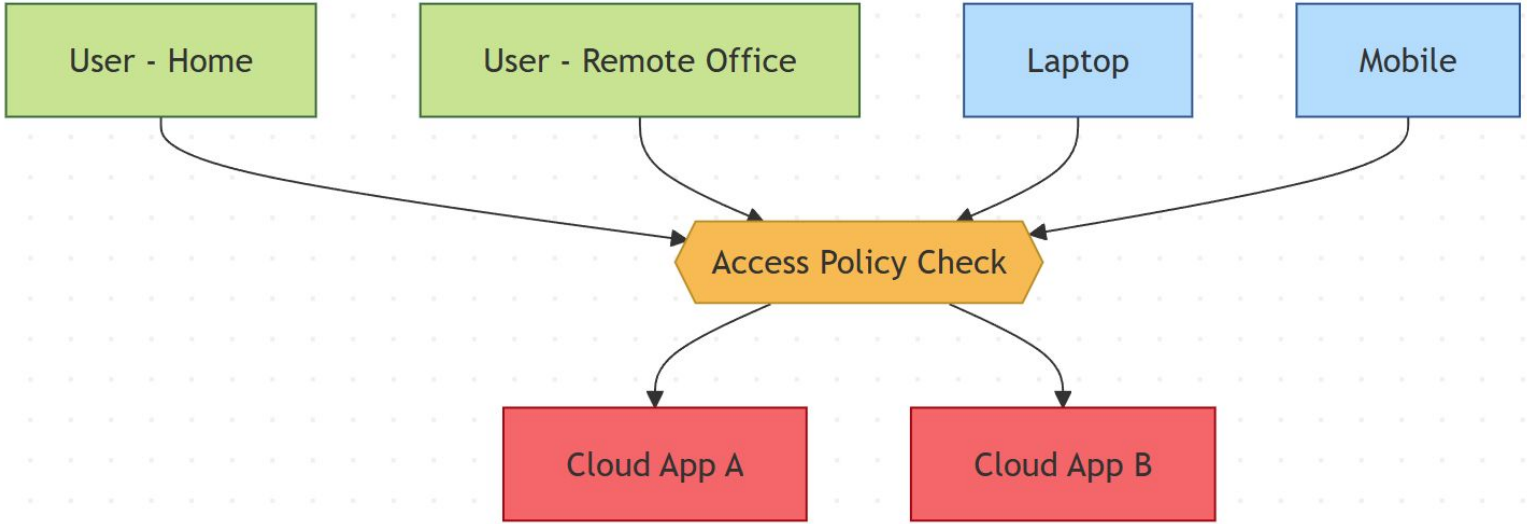
Threat Modeling: Thinking Like an Attacker

- Identify assets, entry points, threats
- Use frameworks like STRIDE, DREAD
- Helps prioritize risks and mitigation



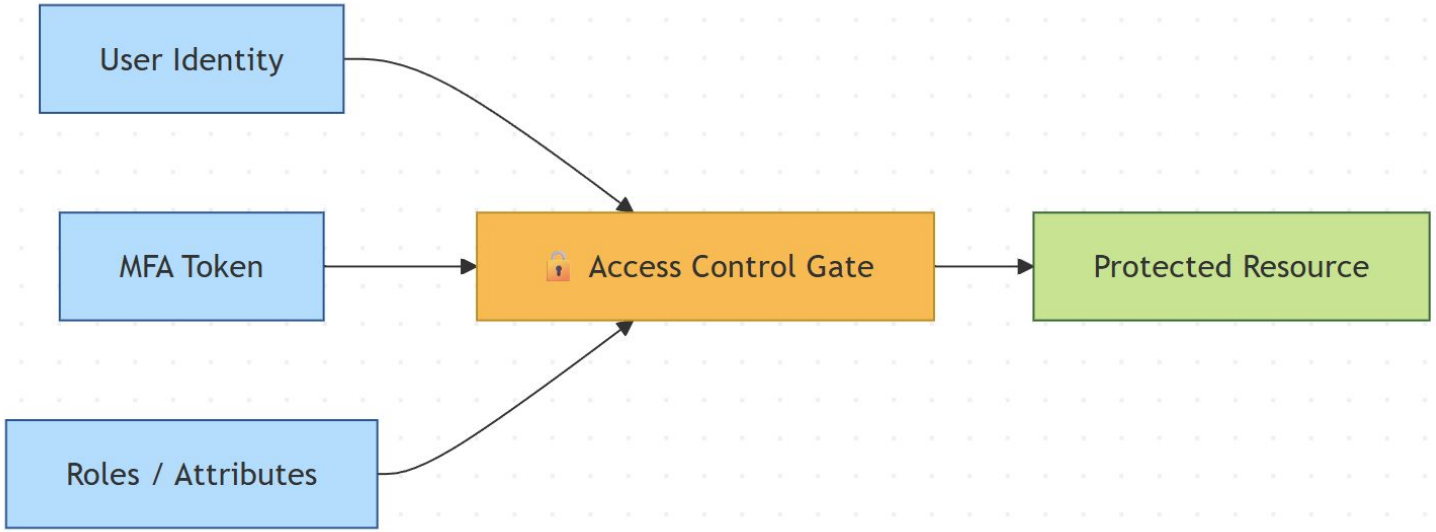
Perimeterless Trust: Why the Edge is Everywhere

- Cloud, APIs, mobile = no fixed edge
- Internal networks not inherently safe
- Secure every interaction, not just the perimeter



Identity as the New Perimeter

- Identity is the foundation of access
- Multi-factor authentication (MFA) is key
- Role-Based (RBAC) & Attribute-Based (ABAC) access controls



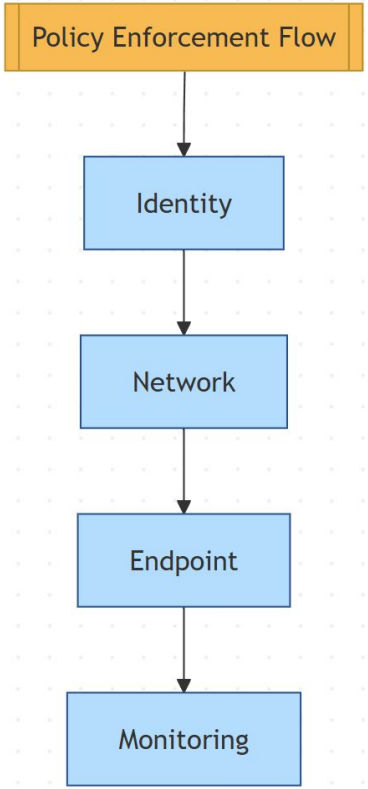
Cloud-Native Security Needs

- Dynamic workloads (containers, serverless)
- Infrastructure as Code (IaC) requires policy-as-code
- Visibility & logging are critical



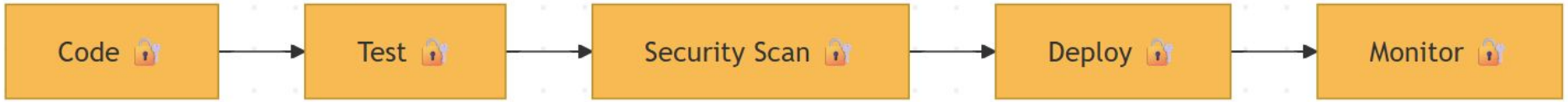
Zero Trust Implementation Building Blocks

- Identity & access management (IAM)
- Network microsegmentation
- Continuous monitoring & analytics
- Device security & posture checks



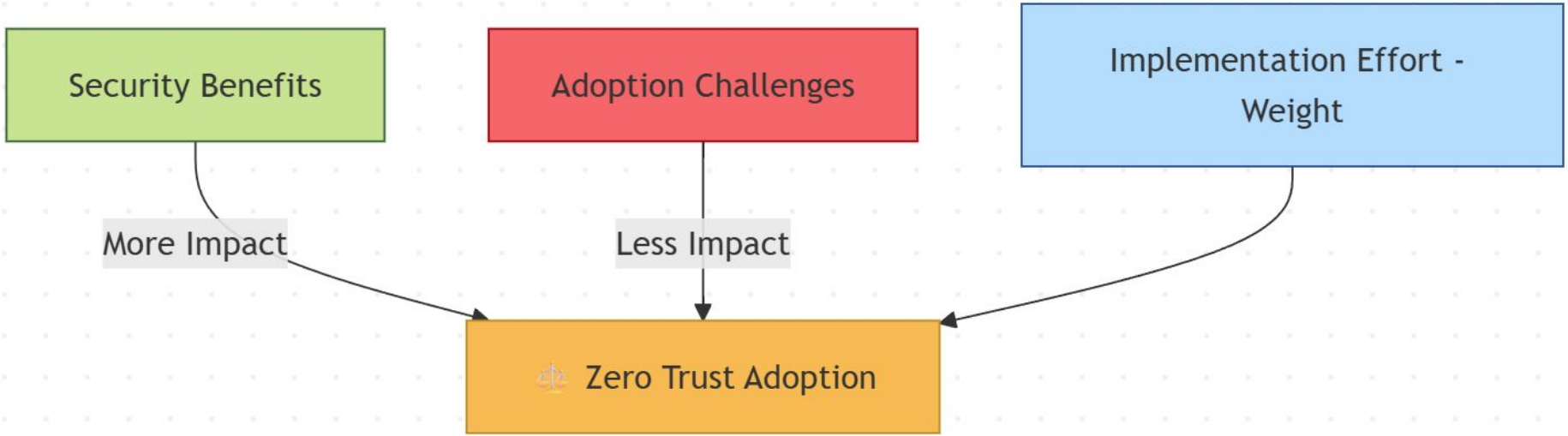
Security in a DevSecOps Culture

- Embed security in development lifecycle
- Shift-left with early security tests
- Secure CI/CD pipelines and secrets



Challenges in Zero Trust Adoption

- Legacy systems and cultural resistance
- Complex identity management
- Requires strong visibility & automation



Summary - Security Principles & Zero Trust

- Zero Trust = mindset + architecture
- Identity and least privilege are central
- Secure everything — always verify
- What's next:
  - Auth & Identity: OAuth2 and OpenID Connect

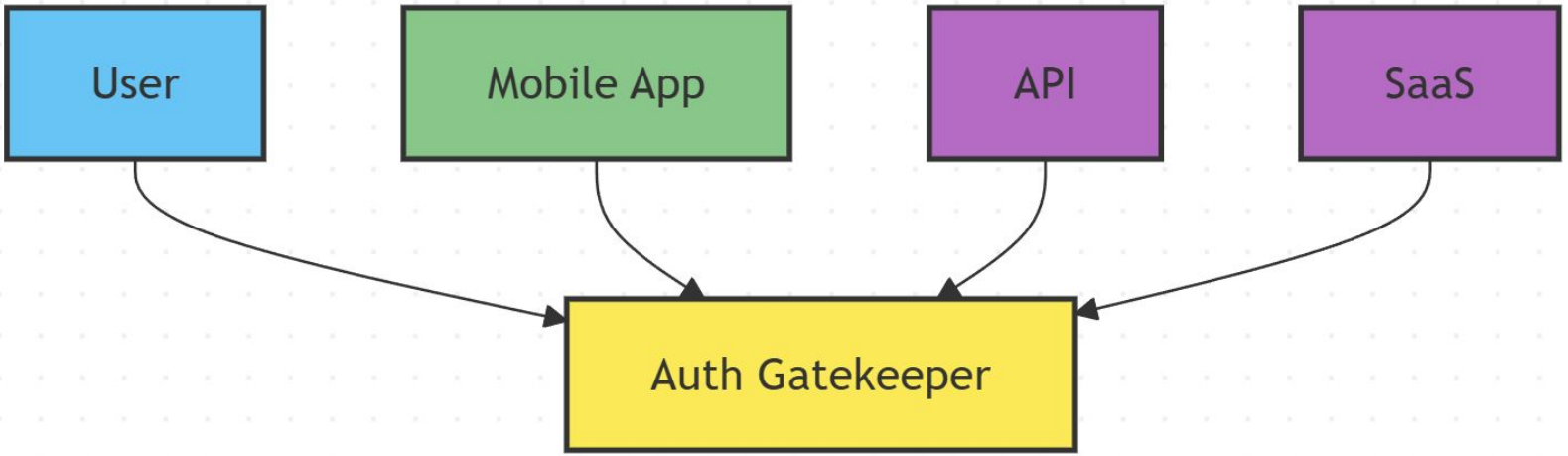
Auth & Identity: OAuth2 and OpenID Connect

Security & Governance in Architecture

Mastering Software Architecture Patterns and System Design

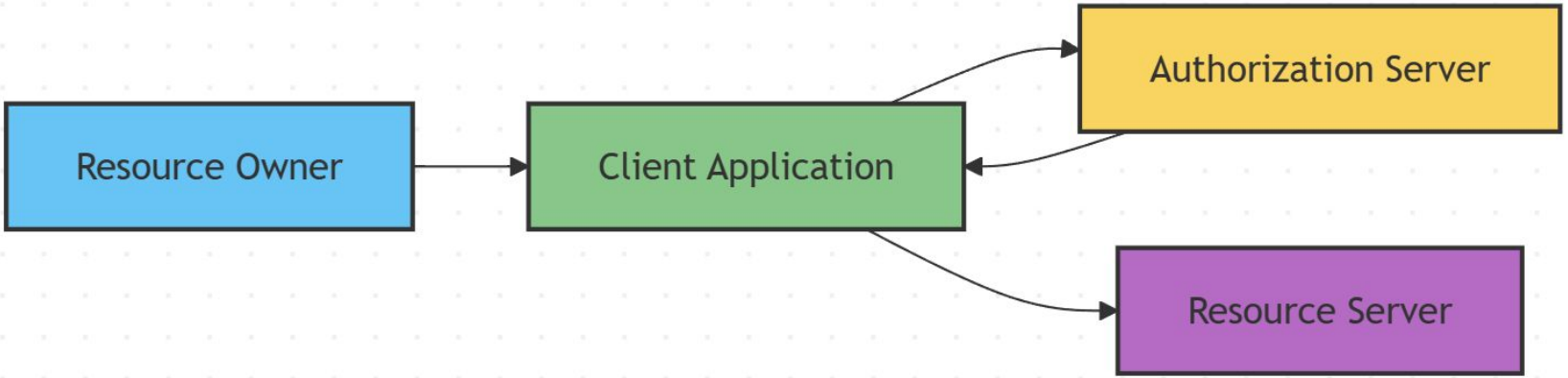
Why Auth & Identity Matter

- Users expect seamless, secure sign-in everywhere
- Microservices & APIs need delegated, granular access
- Compliance (GDPR, HIPAA…) demands robust identity controls



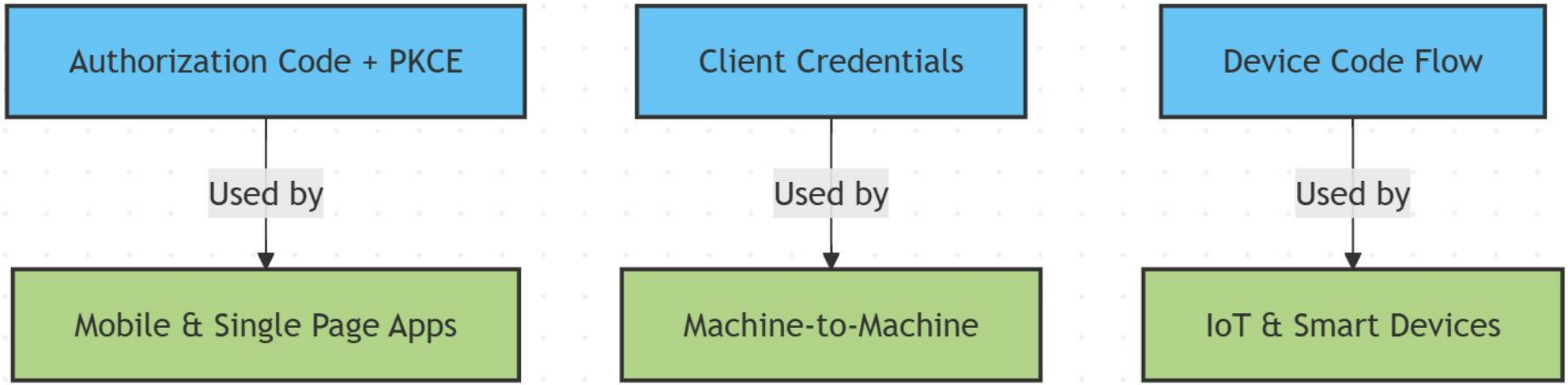
OAuth 2.0 in One Minute

- Delegated authorization protocol (not authentication)
- Four roles: Resource Owner, Client, Authorization Server, Resource Server
- Issues access tokens to the client on the user's behalf



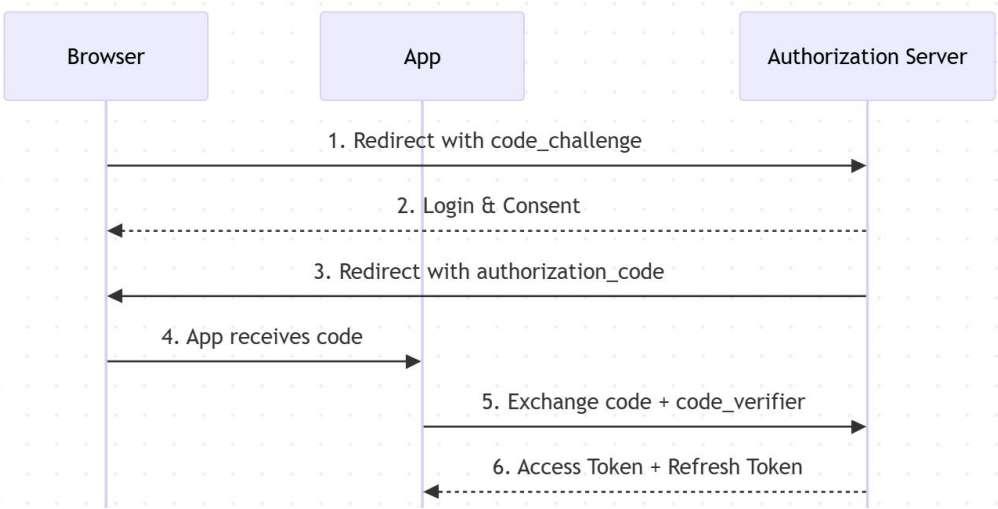
Core OAuth 2.0 Flows

- Authorization Code + PKCE (mobile & SPA)
- Client Credentials (machine-to-machine)
- Device Code (TVs, IoT)



Step-by-Step: Authorization Code + PKCE

- App generates code verifier & code challenge
- User authenticates in system browser
- App receives authorization code

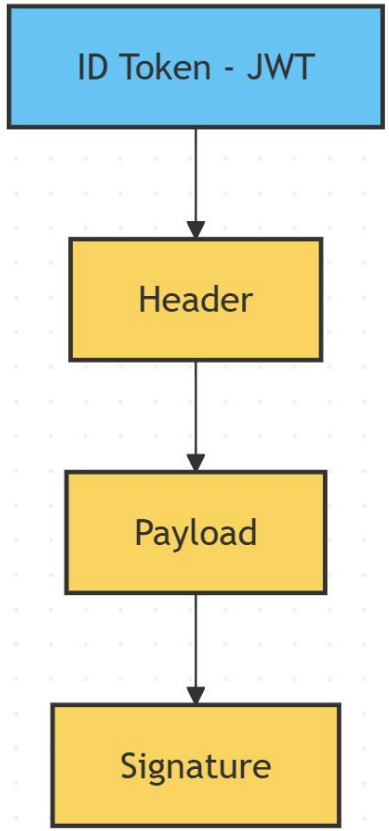


Tokens in the Wild

- Access token – scoped, short-lived
- Refresh token – long-lived, rotates
- ID token (OIDC) – who the user is

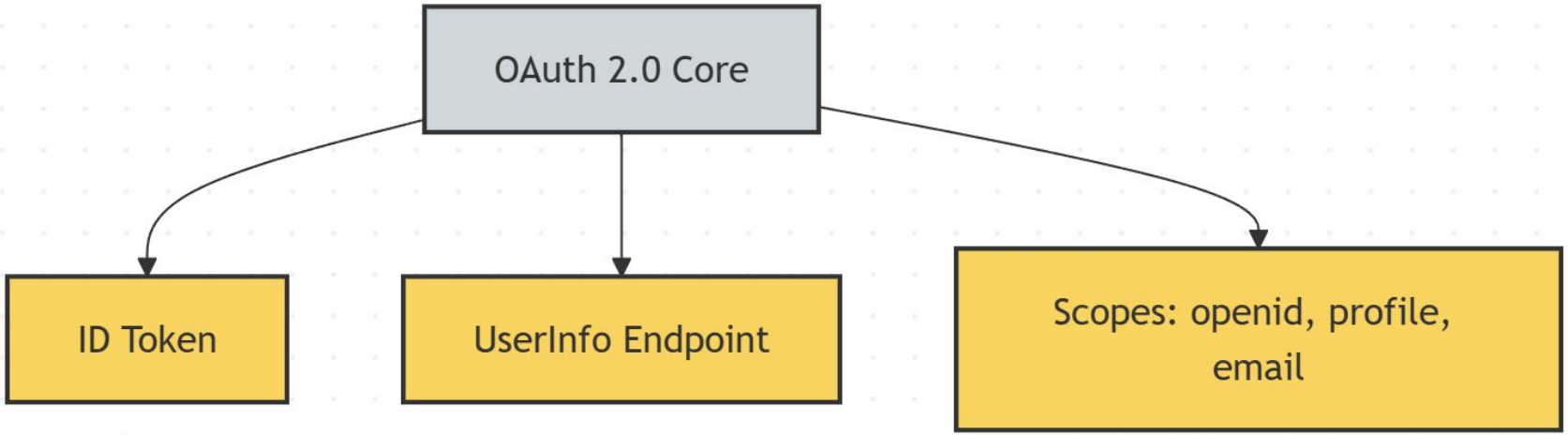
Access Token

Refresh Token



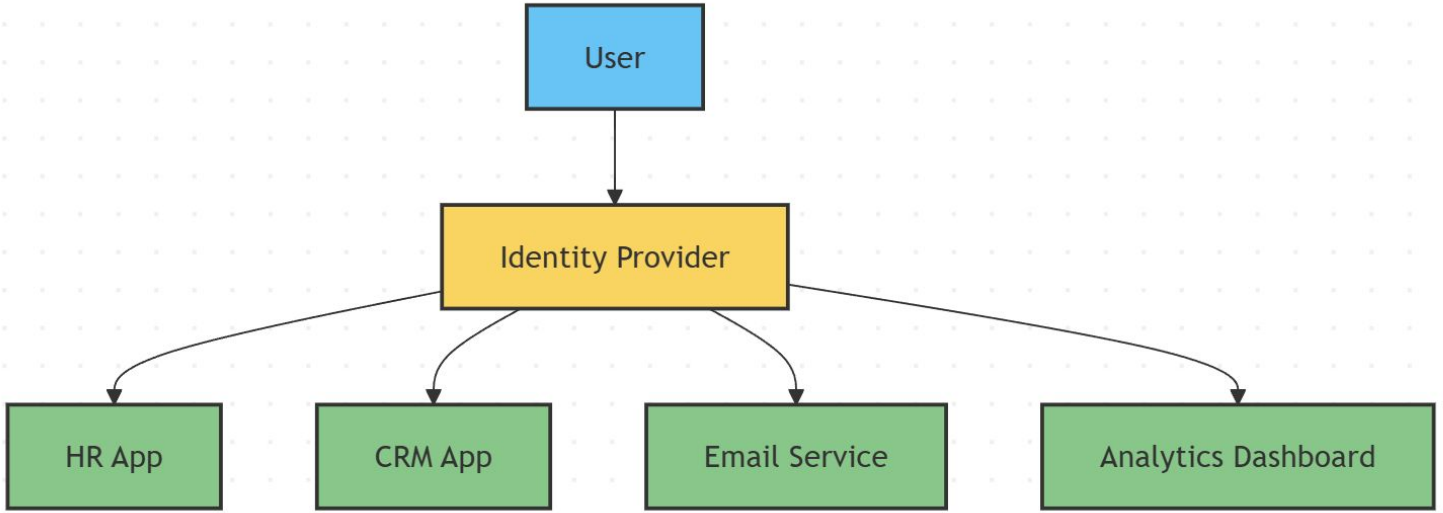
Enter OpenID Connect (OIDC)

- Identity layer on top of OAuth 2.0
- Adds /userinfo endpoint & ID tokens
- Standard scopes (openid, profile, email)



Identity Federation & Single Sign-On

- Trust between Identity Provider (IdP) & Service Provider
- SSO across multiple apps (SAML vs OIDC)
- Just-in-time (JIT) provisioning & centralized logout

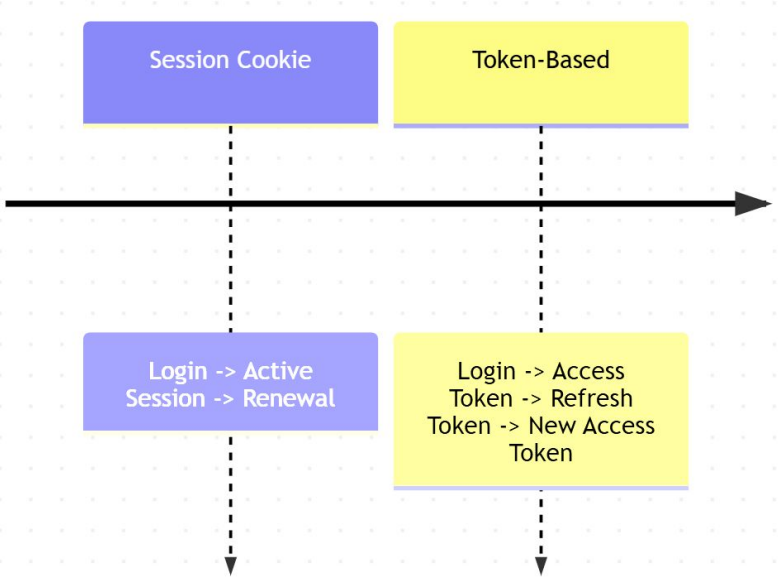


RBAC vs ABAC: Access Control Models

- RBAC (Role-Based Access Control):
  - Users are assigned roles
  - Roles map to a set of permissions
  - Easy to implement and audit
  - Good for static, well-defined roles (e.g., Admin, Editor)
- ABAC (Attribute-Based Access Control):
  - Access based on user, resource, and environment attributes
  - Enables fine-grained, dynamic policies
  - Scales better for complex scenarios (e.g., “Department = HR AND Time < 6PM”)
  - More powerful but harder to manage
- When to Use Which:
  - RBAC → simplicity & auditability
  - ABAC → flexibility & dynamic control
  - Many real-world systems combine both (RBAC + ABAC hybrid)

Session vs Token Lifecycle

- Browser sessions = server-stored state (cookie)
- Token-based = stateless, expires & refreshes
- Trade-offs: revocation, scalability, CSRF/XSS risks



Best Practices & Pitfalls

- Always use HTTPS and PKCE for public clients
- Short token TTL + automatic refresh rotation
- Store tokens in memory or secure cookies, not localStorage
- Implement token introspection / revocation endpoints
- Monitor & log auth events (anomaly detection)

Summary - Auth & Identity: OAuth2 and OpenID Connect

- OAuth 2.0 ⇒ authorization, OIDC ⇒ authentication
- Pick the right flow for the client & threat model
- Design token lifecycles with rotation & revocation in mind
- Combine RBAC & ABAC for real-world complexity
- What's next:
  - API & Service Security

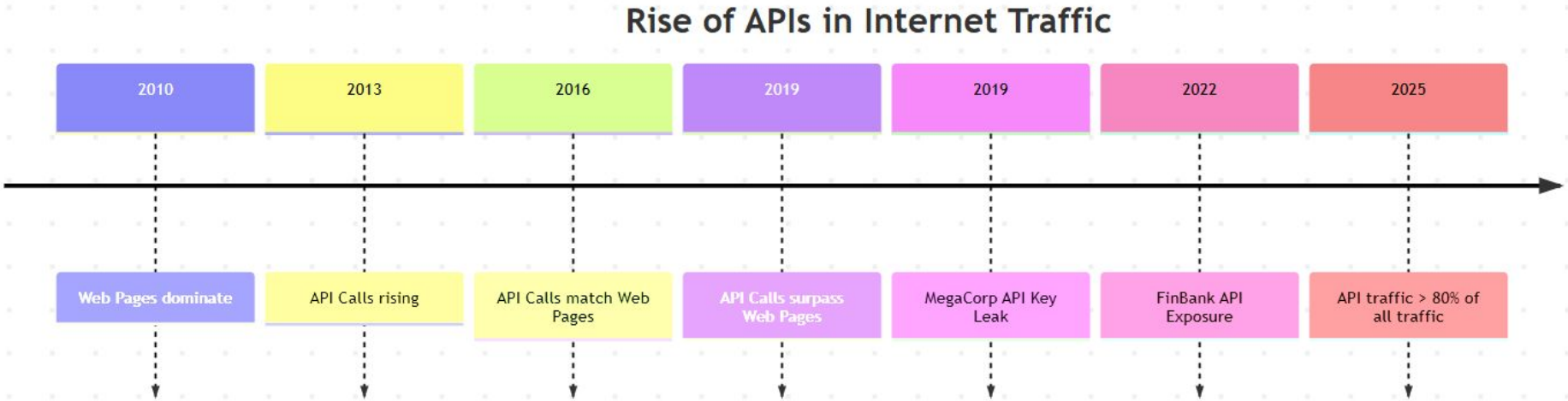
API & Service Security

Security & Governance in Architecture

Mastering Software Architecture Patterns and System Design

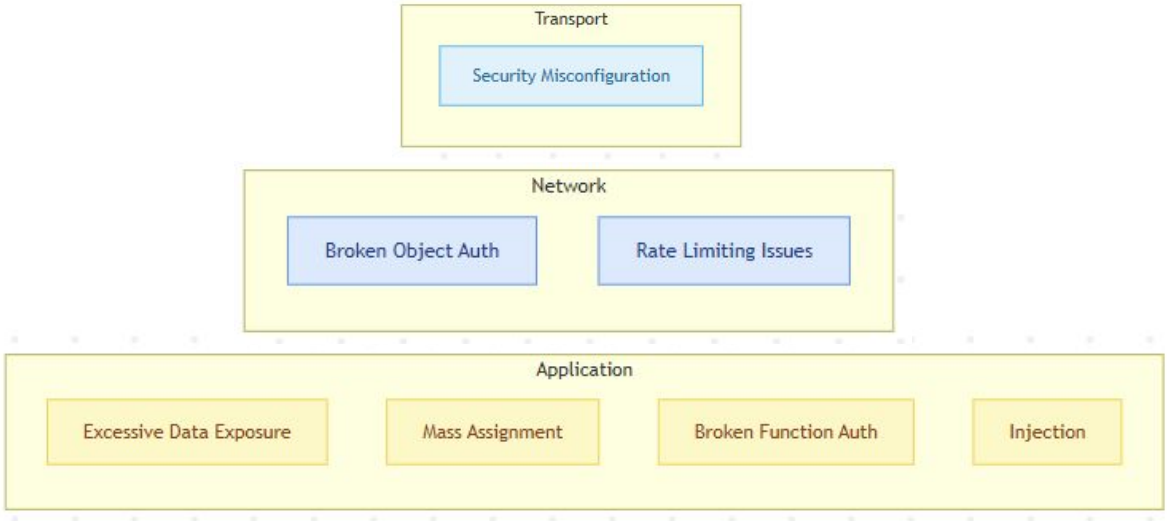
The Rising Stakes of API Security

- APIs now power >80 % of internet traffic
- Data, money, and brand trust flow through every call
- A single leaked token can open your entire backend



Know Your Enemy: Common API Threats

- OWASP API Top 10: Broken auth, excessive data exposure, rate abuse
- Replay & man-in-the-middle attacks on unsecured channels
- Insider or partner misuse of legitimate credentials



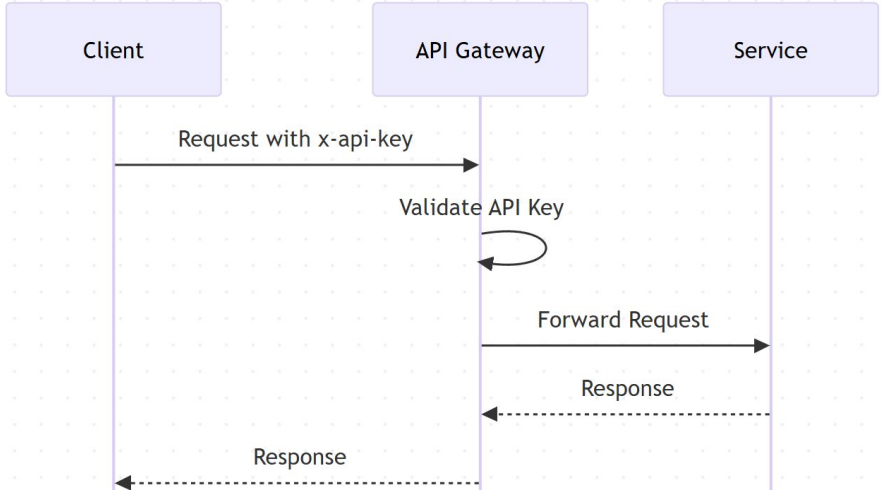
Authentication ≠ Authorization

- Who are you? → AuthN (identity)
- What may you do? → AuthZ (permissions)
- Both needed for a secure call path



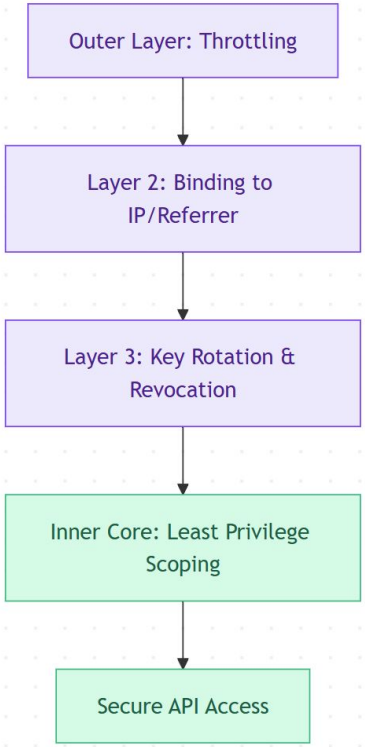
API Keys – Simple Token Auth

- Static, random strings passed in header/query
- Great for server-to-server or anonymous read APIs
- Easy to provision & rotate



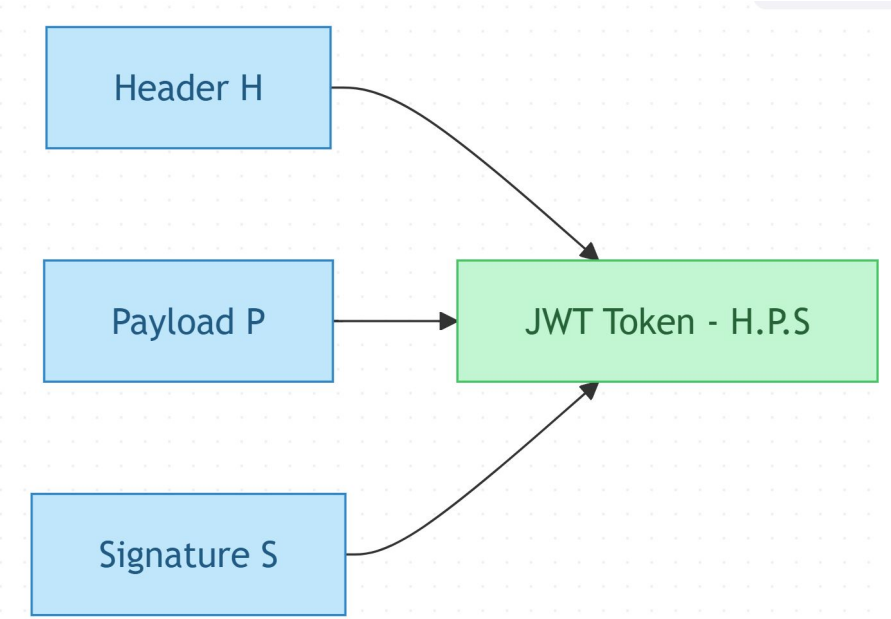
Hardening API Keys in Practice

- Least scope: per-service or per-method keys
- Rotate & revoke: automate via CI/CD
- Bind to IP, referrer, or device fingerprint
- Throttle: pair with rate limiting



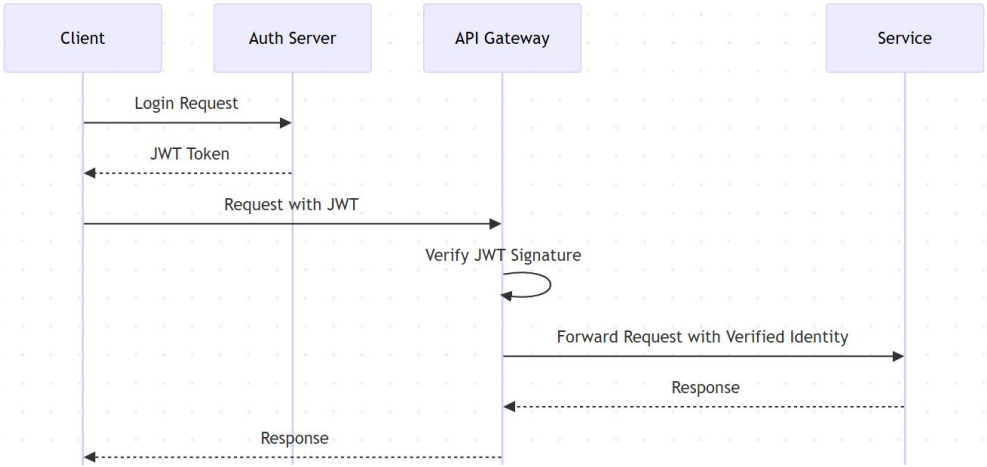
JWT – Compact Claims Token

- Three Base64 parts: header, payload, signature
- Self-contained: no DB lookup needed
- Used for user & service authentication, authZ claims



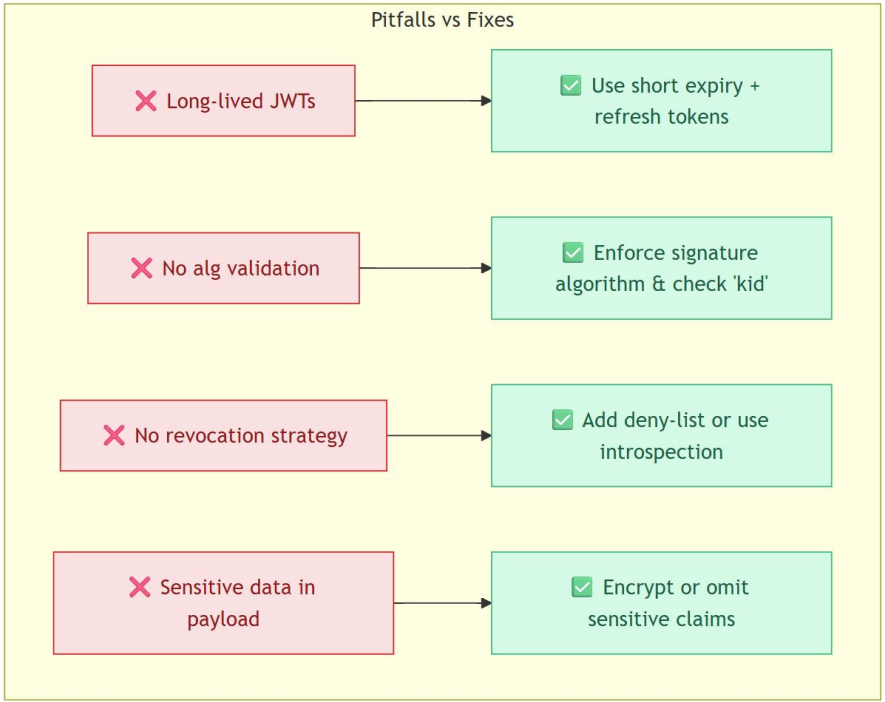
JWT in Action: End-to-End Flow

- User logs in → Auth Server signs JWT
- Client stores & sends token with each call
- Gateway verifies signature, expiry, audience
- Downstream services trust the assertions



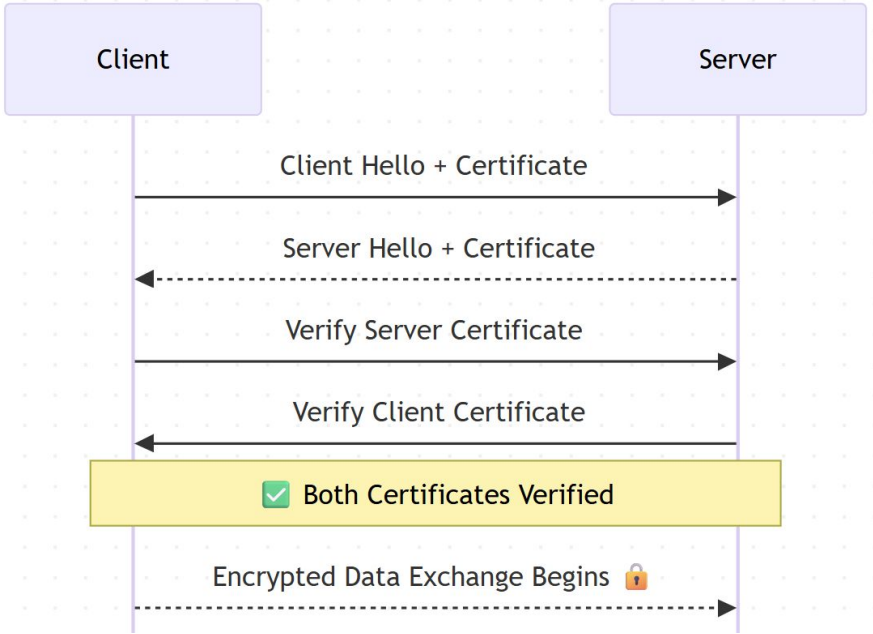
Securing JWTs: Pitfalls & Mitigations

- Short exp + refresh tokens → limit replay window
- kid header validation to block alg-none downgrade
- Server-side deny-list / introspection for instant revocation
- Sign & encrypt sensitive claims



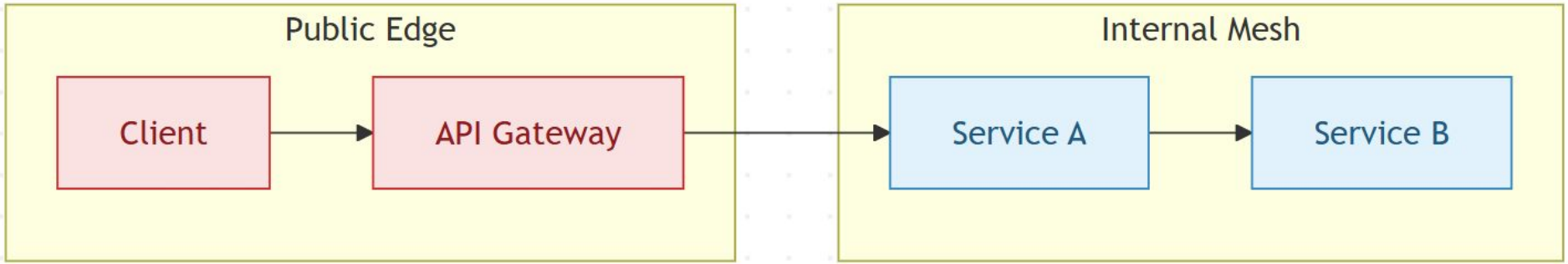
Mutual TLS (mTLS): Service-to-Service Trust

- Both client and server present X.509 certs
- TLS handshake performs identity + key exchange
- Ideal inside zero-trust meshes (Istio, Linkerd)



Internal vs External Interfaces

- External: Public docs, stricter rate limits, OAuth2/JWT
- Internal: mTLS, service mesh policies, private DNS
- Shared: Central gateway for logging & threat mitigation



Defense in Depth & Key Takeaways

- Layer controls: key/JWT at edge, mTLS inside, DDoS & WAF in front
- Monitor: structured logs, anomaly alerts, trace IDs
- Security is a journey: iterate, test, and rotate!
- What's next:
    - Secrets & Token Management

Secrets & Token Management

Security & Governance in Architecture

Mastering Software Architecture Patterns and System Design

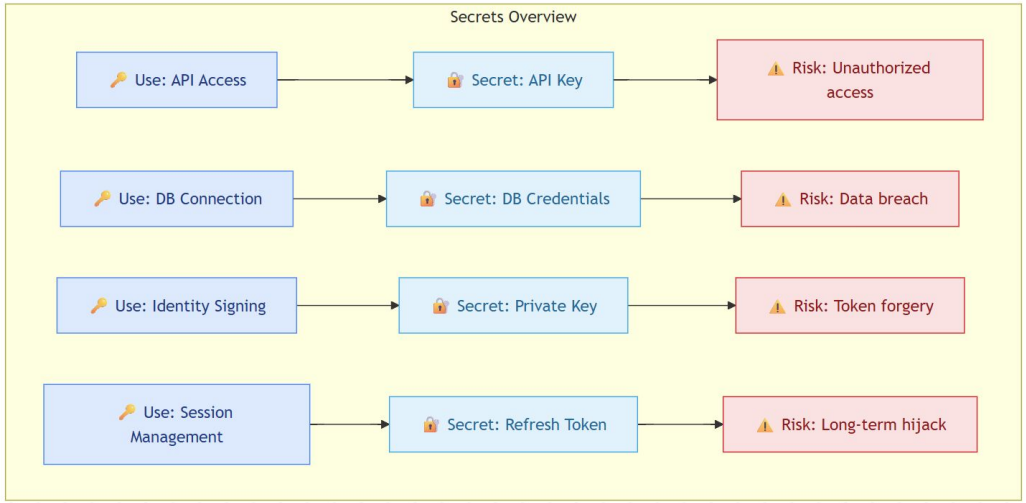
Why Secrets Management Matters

- Secrets = credentials, API keys, tokens, private keys
- Leaked secrets = instant compromise
- Manual handling = human error



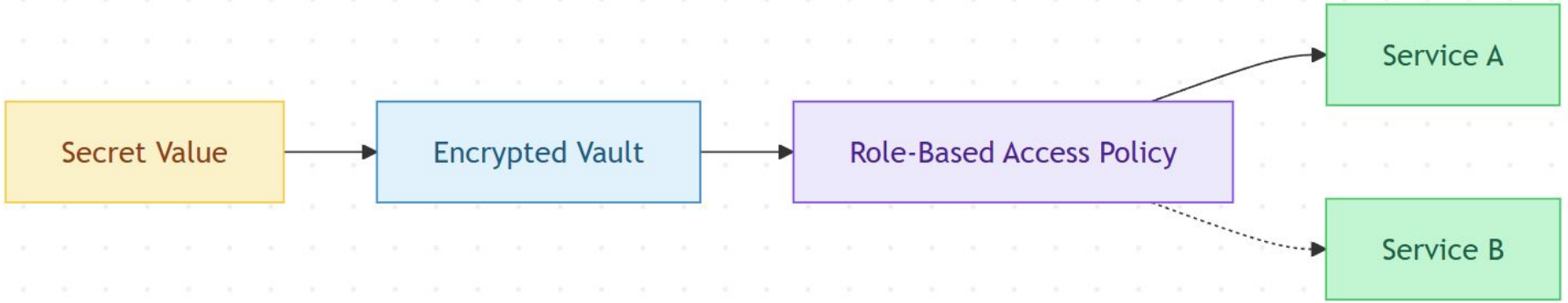
Types of Secrets in Systems

- API keys, DB credentials
- Signing keys & certificates
- Access tokens, refresh tokens
- Encryption keys for data at rest



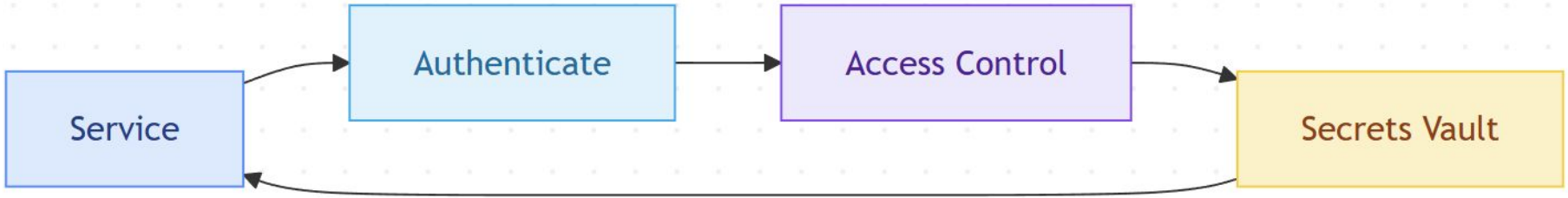
Principles of Secure Secret Storage

- Never store secrets in code or repo
- Encrypt at rest + restrict access
- Use secrets management services



Vaults & Secret Stores

- Tools: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault
- Fine-grained access control
- Auto-expire, audit, and rotate



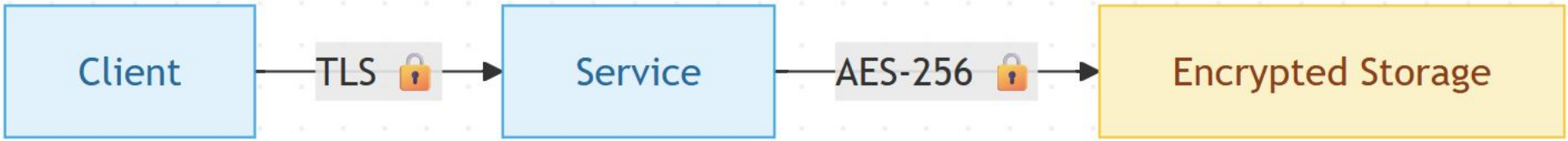
Secret Rotation: Why & How

- Reduces exposure window
- Automate with CI/CD or vault integrations
- Rotate DB passwords, API keys, signing keys



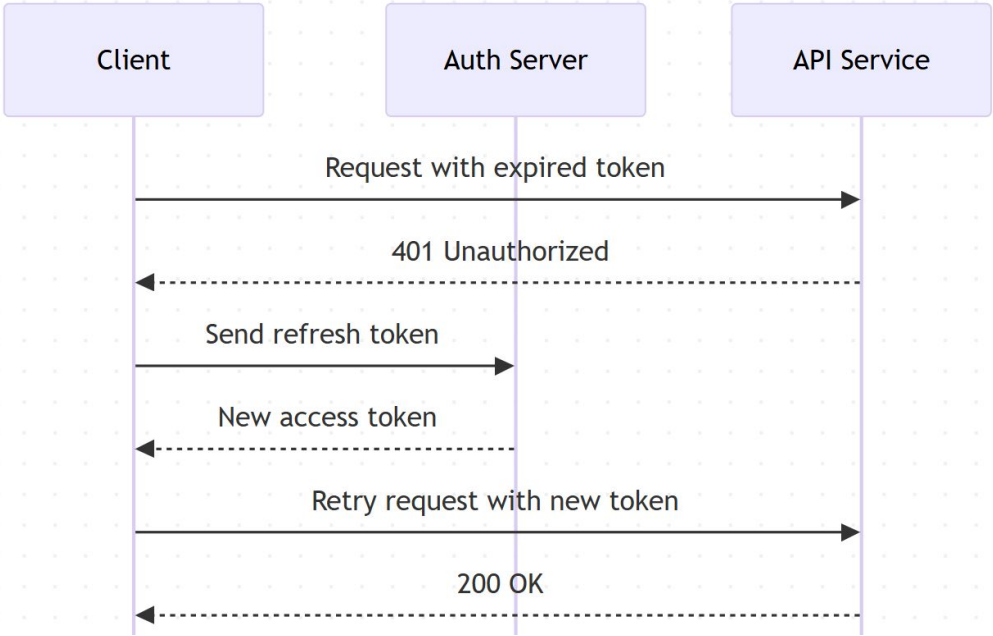
Encrypted Storage & Transit

- Encrypt secrets at rest and in motion
- Use TLS for transport
- Encrypt backing stores (e.g., env vars, config files)



Handling Expiring Tokens

- Access tokens often expire quickly
- App must detect expiration and respond
- Avoid hard failures — refresh silently



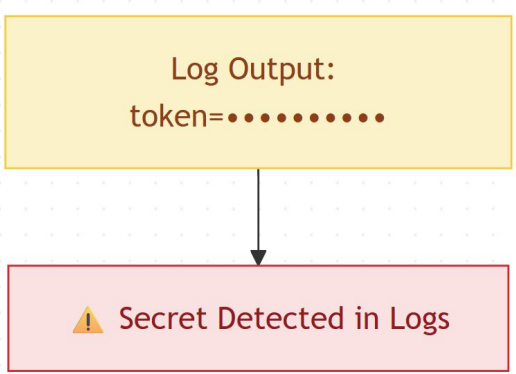
Refresh Tokens & Strategies

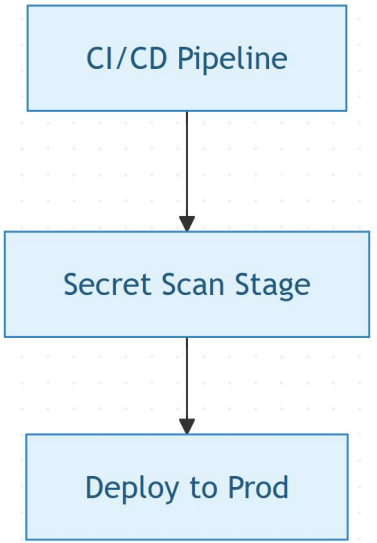
- Long-lived tokens to get short-lived ones
- Secure storage: don’t expose in frontend
- Rotate refresh tokens too (rotate-on-use)



Preventing Secret Leakage

- Block secrets in logs, stack traces
- Scan repos for hardcoded tokens
- Use CI/CD checks to block commits





Key Takeaways - Secrets & Token Management

- Store secrets in vaults, not in code
- Rotate regularly, encrypt always
- Detect leaks early, handle tokens smartly
- What’s Next:
  - Real-World Security Scenarios

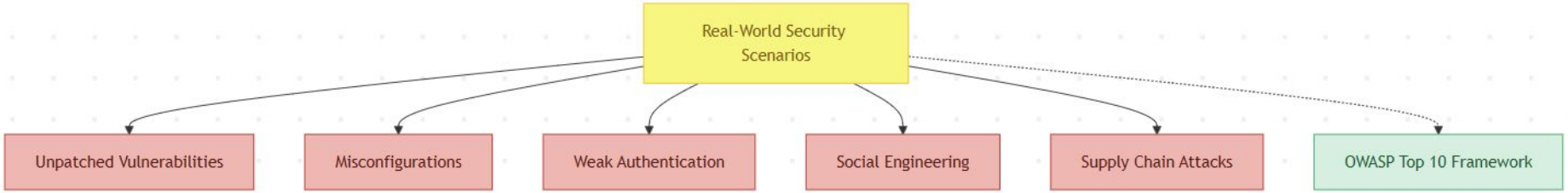
Real-World Security Scenarios

Security & Governance in Architecture

Mastering Software Architecture Patterns and System Design

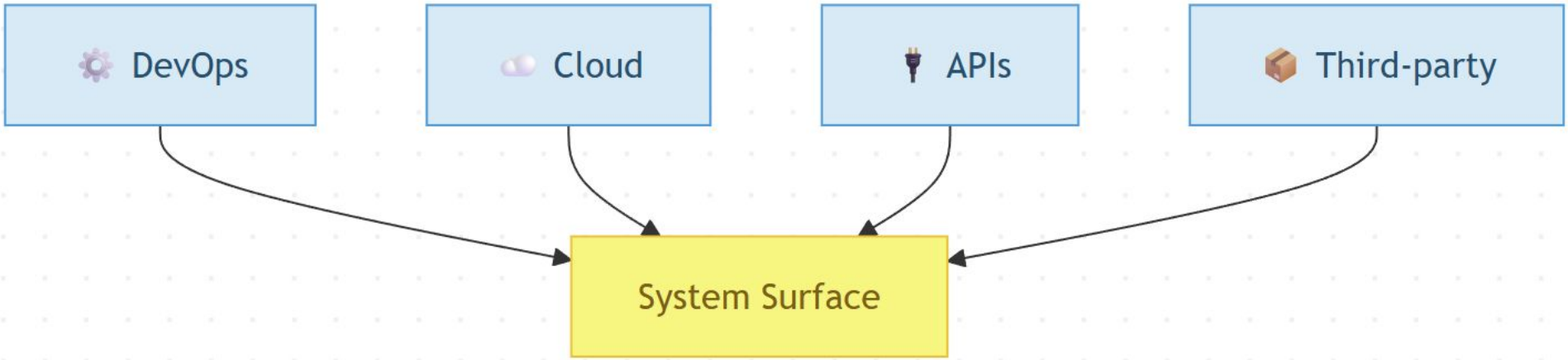
Real-World Security Scenarios

- Threats are not hypothetical — they happen daily
- OWASP Top 10: A practical guide to major risks
- Today’s focus: vulnerabilities, real-world impact, layered defense



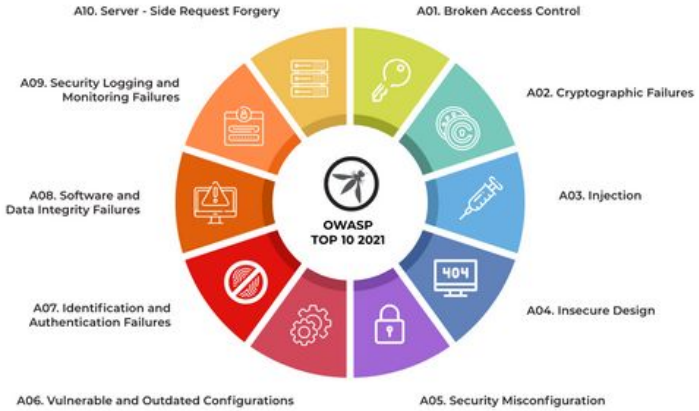
Why Vulnerabilities Persist

- Software complexity and human error
- Rapid deployments with security lag
- Misunderstanding of risk priorities



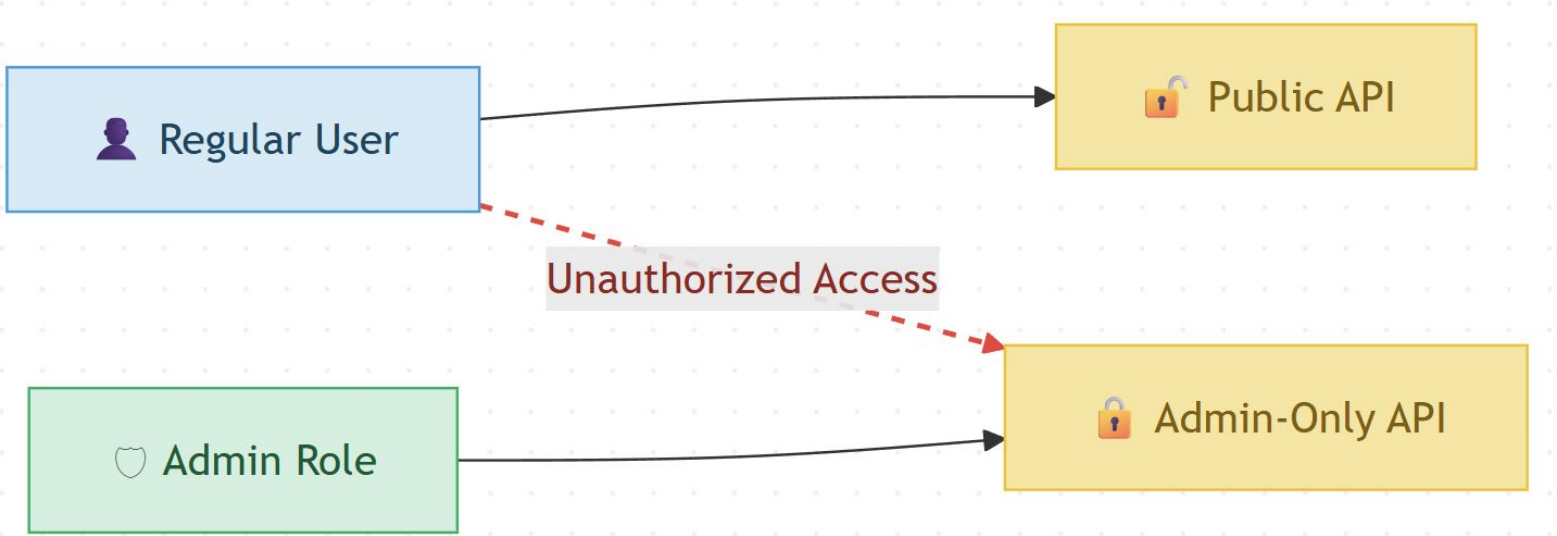
OWASP Top 10 – Overview

- Industry-standard list of critical web app security risks
- Updated regularly based on data + expert input
- Not just a checklist — a mindset for secure development



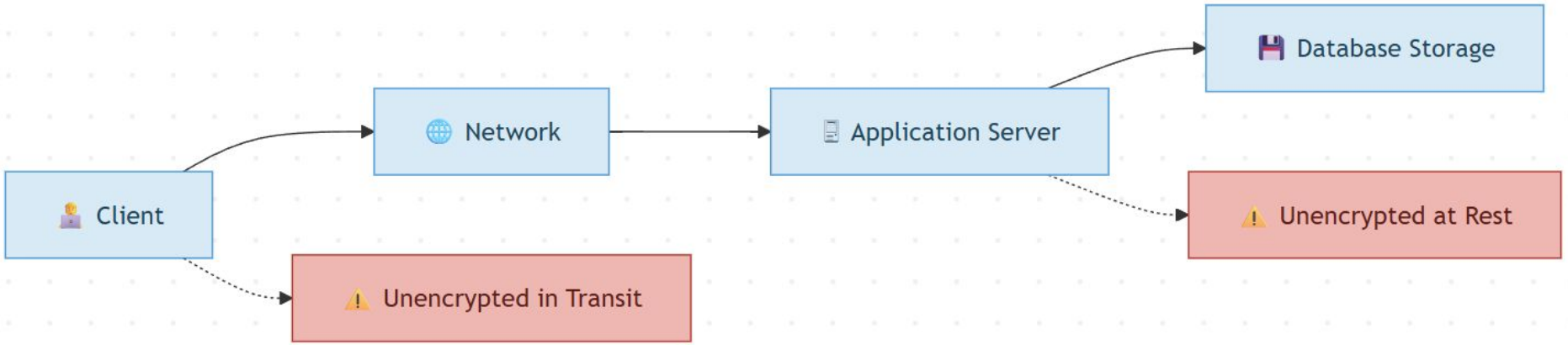
A01 – Broken Access Control

- Users access data or functions beyond their role
- 94% of tested apps had access control issues
- Real-world: Data leaks, privilege escalations



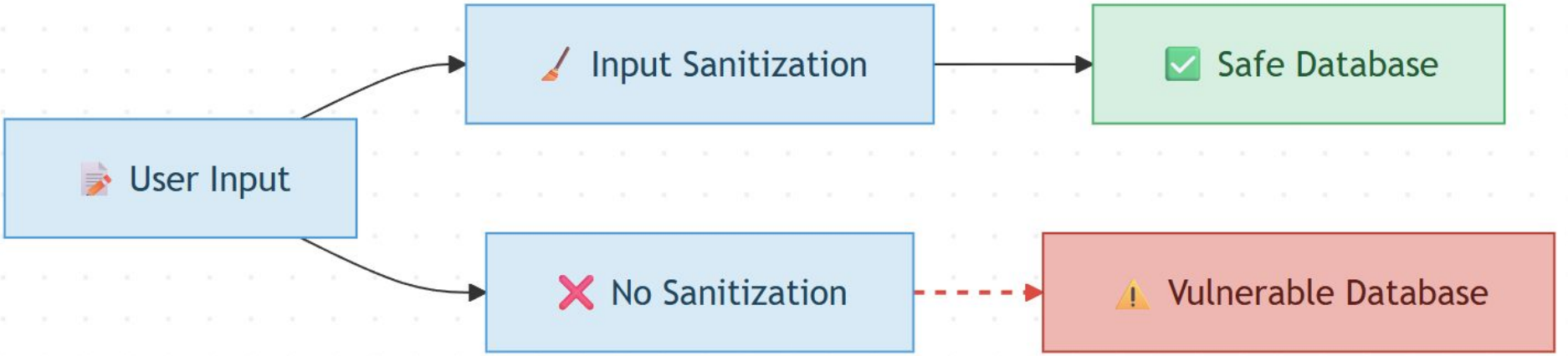
A02 – Cryptographic Failures

- Insecure data storage, weak or no encryption
- Includes outdated TLS, exposed secrets
- Real-world: Credential theft, data tampering



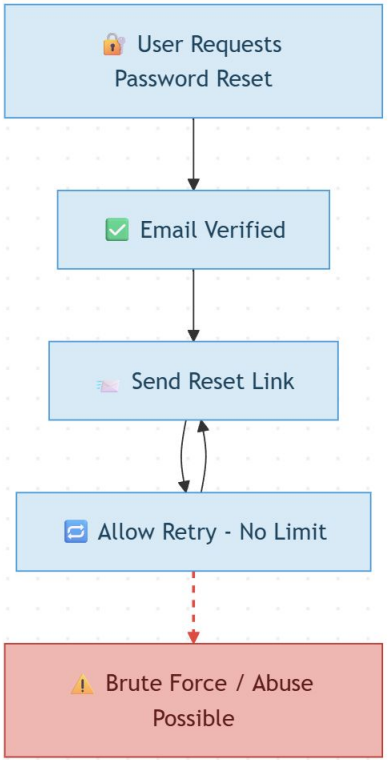
A03 – Injection

- User input interpreted as commands (SQL, LDAP, NoSQL)
- Still prevalent despite years of awareness
- Real-world: DB dumps, data modification, RCE



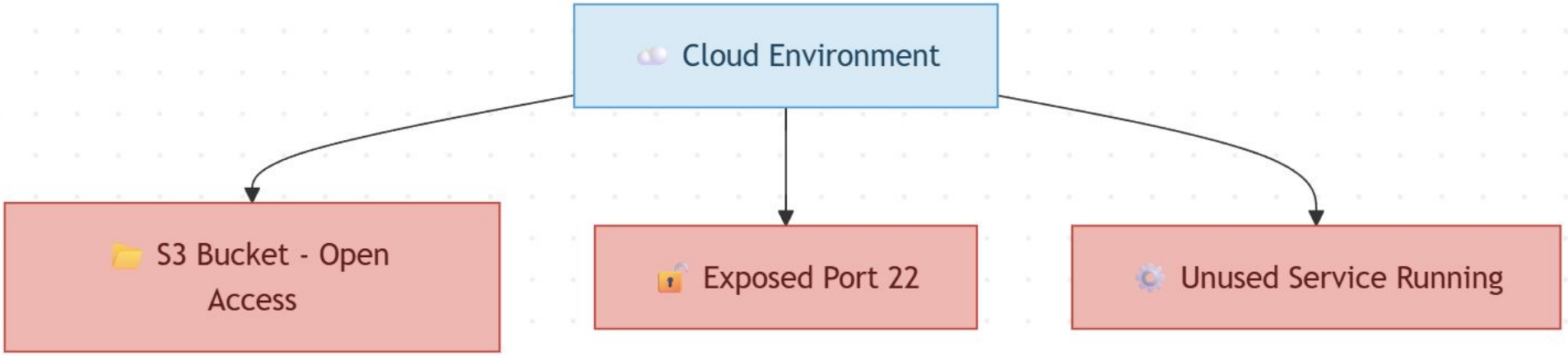
A04 – Insecure Design

- Flawed architecture — not just code bugs
- Example: No rate limiting, unsafe workflows
- Real-world: Business logic abuse, scraping, abuse



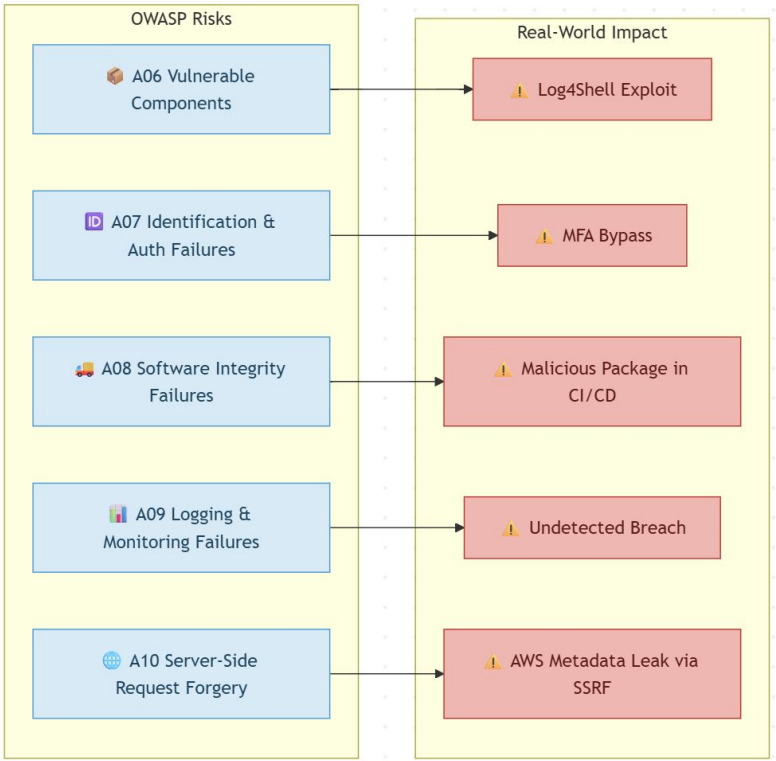
A05 – Security Misconfiguration

- Default credentials, open ports, verbose error messages
- Common in cloud, containers, and CI/CD pipelines
- Real-world: Unintended public exposure



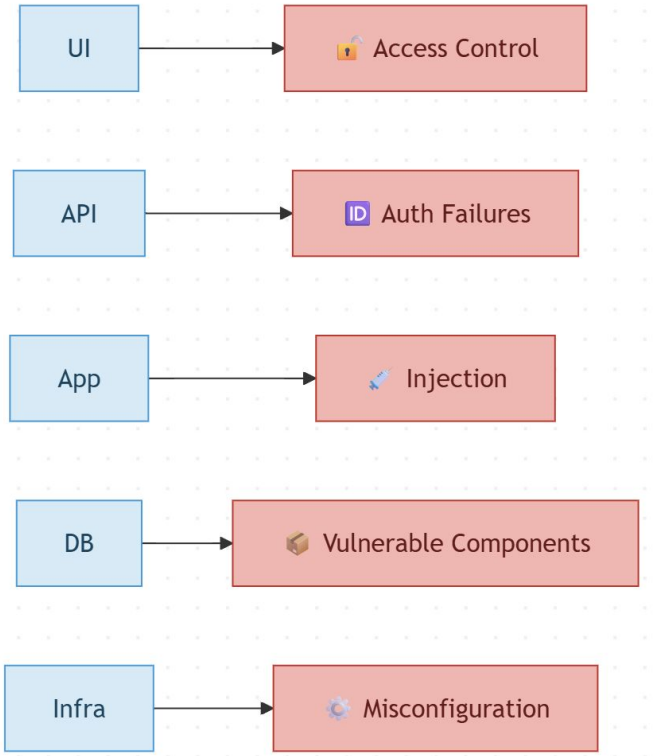
A06–A10 – The Remaining Risks

- A06: Vulnerable Components → unpatched libraries
- A07: Identification Failures → weak auth, poor session handling
- A08: Software Integrity → CI/CD & supply chain trust
- A09: Logging & Monitoring → blind to breaches
- A10: SSRF → attackers make server callouts



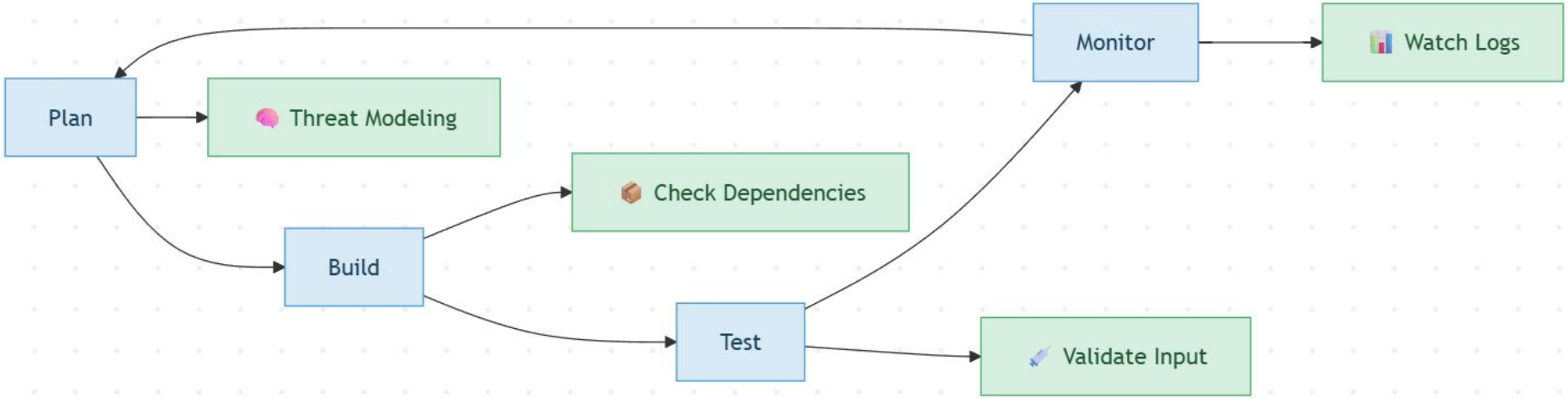
OWASP + Layered Security

- OWASP risks exist across all layers
- Design defenses across: UI, API, storage, network
- Combine: validation, RBAC, encryption, logging



From OWASP to Action

- Use OWASP as review lens in every sprint
- Perform threat modeling aligned to Top 10
- Integrate into CI/CD security scans



Summary - Real-World Security Scenarios

- OWASP = practical framework for security-first design
- Top risks align with real-world breaches
- Prevention starts at design, not just in code
- What's next:
  - Event-Driven & Reactive Systems